Assignment #5 - Strategy Pattern

Option A: Navigation / Route Planning

Name: Serekpayev Ali

1. Introduction

I chose Option A: Navigation / Route Planning. The program is a simple navigator that can build a route in different ways: by car, on foot and by bicycle. The start point, the destination and the distance are the same, but the route and the travel time are different for each way of moving.

The Strategy pattern fits this task well. Each way of building a route is a separate algorithm, and all algorithms have the same interface `RouteStrategy`. The class `Navigator` (the Context) stores the chosen strategy and calls it. Because of this, the user can change the transport at runtime with `setStrategy()` and does not need to create a new navigator.

Without the pattern, `Navigator` would have a big `if / else` or `switch` with the transport type inside. Every new transport would force us to edit this class. With Strategy, we only add a new class.

The project has six Java files in the package `navigation`: `RouteStrategy`, `CarStrategy`, `WalkStrategy`, `CycleStrategy`, `Navigator` and `Main`. Each strategy prints its own route and calculates the estimated time using its own speed (car 60 km/h, walking 5 km/h, cycling 15 km/h).

2. UML Class Diagram

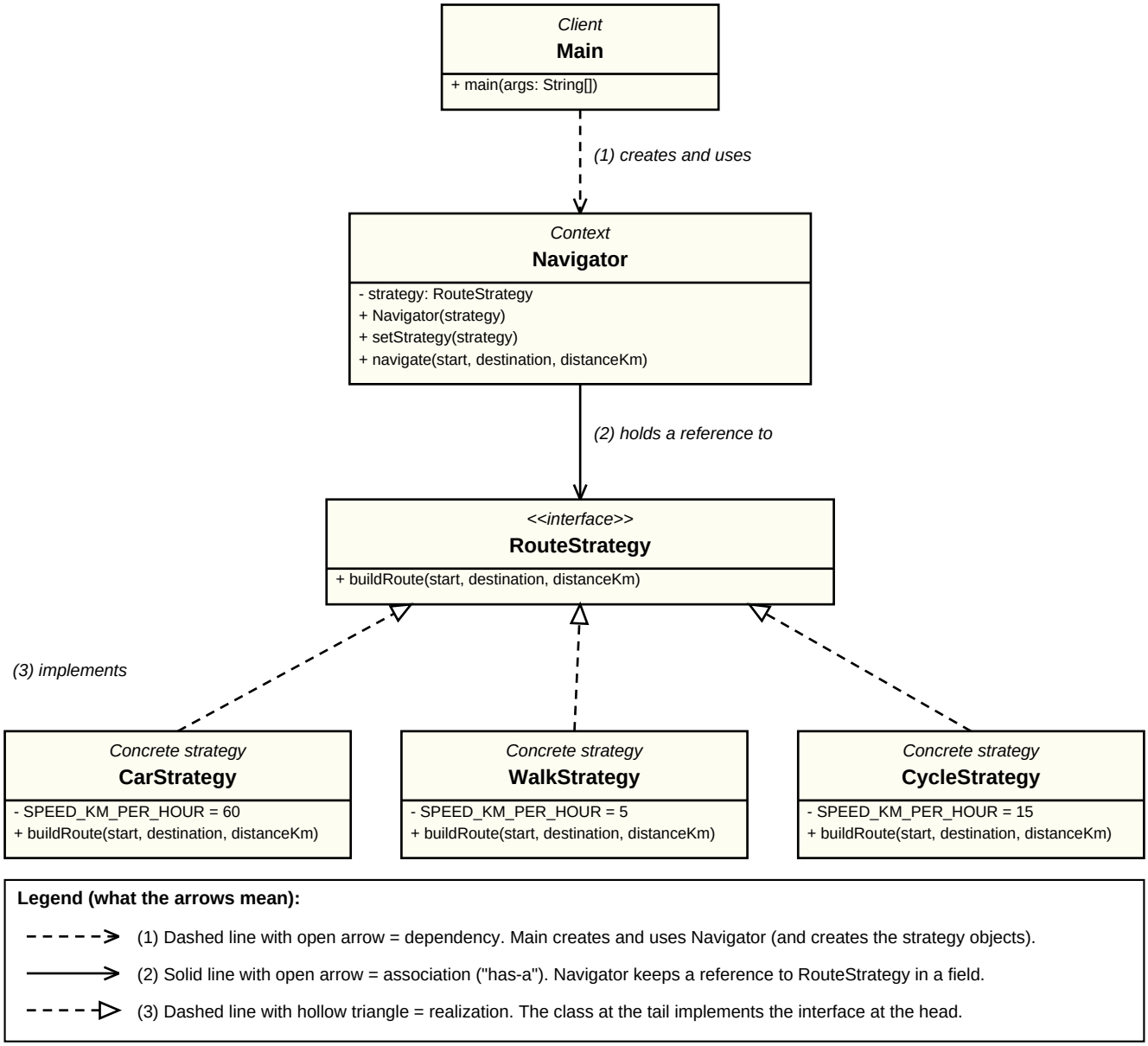

Figure 1. Class diagram of the navigation system (Strategy pattern)

How to read the diagram

Roles in the pattern. `Main` is the Client. `Navigator` is the Context. `RouteStrategy` is the Strategy interface. `CarStrategy`, `WalkStrategy` and `CycleStrategy` are the Concrete Strategies.

Arrow (1), dashed with an open arrowhead (dependency): `Main` creates a `Navigator` and calls its methods. `Main` also creates the strategy objects with `new CarStrategy()` and others, so it depends on them too.

Arrow (2), solid with an open arrowhead (association): `Navigator` has a private field `strategy` of type `RouteStrategy`. It knows only the interface and not the concrete classes. This is the most important arrow of the pattern.

Arrow (3), dashed with a hollow triangle (realization): `CarStrategy`, `WalkStrategy` and `CycleStrategy` implement `RouteStrategy`. The triangle points to the interface.

Signs in the boxes. + means a public member, - means a private member. The word <<*interface*>> marks an interface. Each box has three parts: name, fields, methods (here fields and methods are in one list to save space).

3. Clean Code Principles

3.1 Polymorphism Instead of Conditionals

`Navigator` does not check which transport is selected. It just calls `buildRoute()` through the interface, and Java runs the needed algorithm. **Before** (bad variant, not used):

```
if (type.equals("car")) { ... }
else if (type.equals("walk")) { ... }
else if (type.equals("cycle")) { ... }
```

After (my code):

```
public void navigate(String start, String destination, double distanceKm) {
    strategy.buildRoute(start, destination, distanceKm);   // no if or switch
}
```

This removes the long condition and the risk to forget one branch.

3.2 Meaningful Names

Names show the purpose of the class or method, so the code is easy to read without comments.

```
class CarStrategy implements RouteStrategy      // route for a car
class WalkStrategy implements RouteStrategy     // route for a pedestrian
void setStrategy(RouteStrategy strategy)        // change the algorithm
void buildRoute(String start, String destination, double distanceKm)
```

3.3 Single Responsibility Principle

Each concrete strategy has only one job: build a route for one kind of transport. `Navigator` only stores and runs the strategy, and `Main` only starts the demonstration. If the speed of a bicycle changes, only `CycleStrategy` must be edited.

```
public class CycleStrategy implements RouteStrategy {
    private static final double SPEED_KM_PER_HOUR = 15;   // only cycling logic here
    ...
}
```

3.4 Encapsulation

The field `strategy` is private, so the client cannot change it directly. It can be changed only through the public method `setStrategy()`. The speed in each strategy is also a private constant.

```
public class Navigator {
    private RouteStrategy strategy;          // hidden from other classes

    public void setStrategy(RouteStrategy strategy) {
        this.strategy = strategy;            // the only way to change it
    }
}
```

3.5 Open for Extension, Closed for Modification

A new transport can be added without changing `Navigator` or the existing strategies. For example, a bus strategy:

```
public class BusStrategy implements RouteStrategy {
    @Override
    public void buildRoute(String start, String destination, double distanceKm) {
        System.out.println("Bus route from " + start + " to " + destination);
    }
}
```

To use it, `Main` only needs `navigator.setStrategy(new BusStrategy());`. `Navigator` stays the same.

Test results

Output of `Main`: the same `Navigator` object builds three different routes.

```
=== Car Strategy ===
Car route from Astana to Borovoe
Route: Highway and main roads.
Distance: 250.0 km
Estimated time: 4 h 10 min

=== Walking Strategy ===
Walking route from Astana to Borovoe
Route: Sidewalks and pedestrian paths.
Distance: 250.0 km
Estimated time: 50 h 0 min

=== Cycling Strategy ===
Cycling route from Astana to Borovoe
Route: Bicycle lanes and cycling paths.
Distance: 250.0 km
Estimated time: 16 h 40 min
```

4. Strategy vs. State

The class diagrams of Strategy and State look almost the same: a Context keeps a reference to an interface, and several classes implement it. The difference is in the idea and in who changes the object.

Strategy. The client chooses an algorithm from outside. In my project the user decides how to travel, and `Main` calls `setStrategy()`. The strategies do not know about each other and never switch themselves. They are different ways to do the same job.

State. The object changes its behavior when its internal state changes, and the states often switch each other. For example, an order can be New, Paid or Shipped, and the next state is chosen by the object itself after an action.

Here Strategy is the right choice because the route algorithm is selected by the user and does not depend on the internal state of `Navigator`.

5. Conclusion

The Strategy pattern made the navigation system flexible and easy to extend. `Navigator` has no conditions for the transport type, each algorithm is in its own class, and a new strategy can be added without editing the old code. The price is that there are more classes in the project, but each of them is small and has a clear responsibility.

I also checked that the strategy really changes at runtime: one `Navigator` object gives three different results after two `setStrategy()` calls.

6. GitHub Repository

https://github.com/Dayglovv/SDPFive